

CONCEPT

What is a CTE?

A **Common Table Expression** (CTE) is a named, temporary result set defined at the top of a SQL statement using the **WITH** keyword. It exists only for the duration of that single query — think of it as giving a name to a subquery so you can reference it cleanly, even multiple times.

Why use CTEs? They break sprawling queries into logical, named steps. Each step is independently readable, testable, and reusable within the same statement. They're the SQL equivalent of well-named functions in application code.

Here's a query *without* a CTE — a nested subquery that's hard to follow:

```
SELECT department, avg_salary FROM ( SELECT department, AVG(salary) AS WITHOUT CTE  
avg_salary FROM employees GROUP BY department ) sub WHERE avg_salary > 75000  
ORDER BY avg_salary DESC;
```

Now the same logic rewritten with a CTE:

```
WITH dept_averages AS ( SELECT department, AVG(salary) AS avg_salary FROM
employees GROUP BY department ) SELECT department, avg_salary FROM
dept_averages WHERE avg_salary > 75000 ORDER BY avg_salary DESC;
```

Both produce identical results, but the CTE version names the intermediate step, making the intent clear at a glance.

STRUCTURE

Anatomy of a CTE

```
WITH cte_name AS ( ← keyword + name + AS -- any valid SELECT ← the CTE body
SELECT ... FROM ... ) ← closing parenthesis – no comma if only one CTE SELECT ...
← the "main" (outer) query FROM cte_name ← reference the CTE like a table WHERE ...;
```

Key rules to remember:

- **WITH** appears **once** at the very start. Multiple CTEs are separated by **commas**, not additional **WITH** keywords.
- A CTE can reference any CTE defined *before* it in the same **WITH** block.
- The main query at the bottom can reference *any* of the defined CTEs.
- CTEs are scoped to the single statement — they vanish after the semicolon.

Multiple CTEs syntax:

```
WITH first_cte AS (...),
     second_cte AS (...),
     third_cte AS (...)
SELECT ... FROM third_cte ...;
```

CTE vs. Temp Table vs. Subquery: A CTE is *not* materialized (in most engines) — it's essentially an inline view. If you need to reference the same heavy computation across multiple separate statements, consider a temporary table instead. Within a single statement though, CTEs win on clarity.

WORKED EXAMPLE

Your First CTE

Scenario: You have a table `orders` with columns `id`, `customer_id`, `amount`, `order_date`. You want to find customers whose total spending exceeds \$1,000.

```
-- Step 1: Define a CTE that totals spending per customer WITH
customer_totals AS ( SELECT customer_id, SUM(amount) AS total_spent FROM
orders GROUP BY customer_id ) -- Step 2: Query the CTE just like a regular
table SELECT customer_id, total_spent FROM customer_totals WHERE total_spent
> 1000 ORDER BY total_spent DESC;
```

What happened:

- `WITH` opens the CTE block. `customer_totals` is the name we chose.
- Inside the parentheses is a complete, valid `SELECT` that groups orders and sums amounts.
- The outer query references `customer_totals` in its `FROM` clause as if it were a real table, then filters and sorts.
- After the semicolon, `customer_totals` ceases to exist — it's not stored anywhere.

PRACTICE

Fill in the Blanks

Each exercise presents a scenario and a CTE query with missing pieces. Type the correct SQL in each blank, then check your work. Use **Hint** if stuck, or **Reveal** to see the answer.

1

Basic CTE — Filter an Aggregation

Easy

Use a CTE to calculate total revenue (price × units_sold) per product, then select only products with revenue above 5000.

products (id, name, category, price, units_sold)

```
with product_revenue as (  
    SELECT name,  
           price * units_sold AS revenue  
    FROM products  
)  
SELECT name, revenue  
FROM product_revenue  
WHERE revenue > 5000  
ORDER BY revenue DESC;
```

Check Answer

Hint

Reveal

Reset

✓ All correct — nice work!

WITH opens the CTE, **AS** binds the name to the subquery, and the outer **FROM** references it. The CTE acts exactly like a temporary table scoped to this single statement.

Create a CTE that computes each customer's order count, then JOIN it to customers to retrieve the customer name and city alongside the count.

```
orders(id, customer_id, amount) & customers(id, name, city)
```

```
WITH order_counts AS (  
    CTE name  
    SELECT customer_id,  
           COUNT(*) AS order_count  
           aggregate...  
    FROM orders  
    GROUP BY customer_id  
)  
SELECT c.name, c.city, oc.order_count  
FROM customers c  
join order_counts oc ON c.id = oc.customer_id  
join ke... join column  
ORDER BY oc.order_count DESC;
```

[Check Answer](#)[Hint](#)[Reveal](#)[Reset](#)

✓ All correct — nice work!

The CTE `order_counts` groups orders to get a count per customer. The outer query JOINS this CTE to `customers` on the shared `customer_id = id` relationship, retrieving human-readable names alongside the computed count.

Define **two CTEs**: the first calculates total sales per region, the second finds the overall average across regions. The outer query selects regions that beat the average.

```
sales (id, rep_id, region, amount, sale_date)
```

```
WITH region_totals AS (  
    SELECT region, SUM(amount) AS total  
    FROM sales  
    GROUP BY region  
)  
    ,  
    separat...  
avg_total AS (  
    SELECT AVG(total) AS avg_val  
    FROM region_totals  
    -- aggrega...  
    -- source CTE  
)  
SELECT rt.region, rt.total  
FROM region_totals rt  
    join  
    -- join type  
    avg_total at ON rt.total > at.avg_val  
ORDER BY rt.total DESC;
```

[Check Answer](#)[Hint](#)[Reveal](#)[Reset](#)

✓ All correct — nice work!

A comma separates the two CTEs (not a second `WITH`). The second CTE `avg_total` queries the first CTE `region_totals` — CTEs can reference earlier siblings. `CROSS JOIN` pairs every region row with the single average row so the outer query can compare them.

4

CTE with a Window Function

Medium

Use a CTE with `ROW_NUMBER()` to rank employees by salary within each department, then select only the top earner per department.

`employees` (id, name, department, salary, hire_date)

```
WITH ranked AS (  
    SELECT name, department, salary,  
           ROW_NUMBER() OVER (  
               window fn  
               PARTITION BY department  
               ORDER BY salary DESC  
           ) AS rn  
    FROM employees  
)  
SELECT name, department, salary  
FROM ranked  
WHERE rn = 1;
```

filter ...

Check Answer

Hint

Reveal

Reset

✓ All correct — nice work!

`ROW_NUMBER()` inside the CTE assigns rank 1 to the highest salary per department (because of `DESC`). The outer query simply filters to `rn = 1`. This is a classic "top-N per group" pattern — one of the most common CTE use cases.

5

Data Cleanup CTE

Medium

First CTE: filter to only 'valid' status readings. Second CTE: compute the average reading per sensor from that clean data. Outer query: show sensors whose average exceeds 100.

sensor_readings (id, sensor_id, reading, recorded_at, status)

```
WITH clean_data AS (  
    SELECT sensor_id, reading  
    FROM sensor_readings  
    WHERE status = 'valid'  
    filter value  
) ,  
  
CTE name sensor_avgs AS (  
    SELECT sensor_id,  
           AVG(reading) AS avg_reading  
    FROM source CTE clean_data  
    GROUP BY sensor_id  
)  
  
SELECT sensor_id, avg_reading
```



```
FROM sensor_avgs

WHERE avg_reading > 100

ORDER BY avg_reading DESC;
```

sort clause

Check Answer

Hint

Reveal

Reset

Answer revealed. Study the pattern, then try the next one!

This pipeline pattern is a CTE superpower: `clean_data` filters junk, `sensor_avgs` aggregates the cleaned rows, and the outer query applies the final business rule. Each step has a clear name and purpose.

6

CTE Feeding an INSERT

Medium

Build a CTE that aggregates sales by month and department, then INSERT those results into the `monthly_summaries` table. CTEs aren't limited to SELECT — they work with INSERT, UPDATE, and DELETE.

```
monthly_summaries(month, department, total_sales) & sales(id, department,
amount, sale_date)
```

```
WITH monthly_agg (
    connect...
    SELECT DATE_TRUNC('month', sale_date) AS month,
           department,
           ???(amount) AS total_sales
    FROM sales
    aggrega...
```

```

GROUP BY DATE_TRUNC('month', sale_date), department
)

??? monthly_summaries (month, department, total_sales)
    DML keyword
SELECT ???
    select cols
FROM monthly_agg;

```

Check Answer

Hint

Reveal

Reset

7

Recursive CTE — Org Chart Hierarchy

Hard

Use a **recursive** CTE to walk an org chart. Start from the CEO (`manager_id IS NULL`) and traverse down, tracking the depth level. Recursive CTEs have two parts: a *base case* (anchor) and a *recursive step* joined with `UNION ALL`.

```
employees (id, name, manager_id (NULL for CEO))
```

```

WITH ??? org_tree AS (
    recursive kw
    -- Anchor: the root (CEO)

    SELECT id, name, manager_id, 0 AS depth
    FROM employees
    WHERE manager_id IS NULL

    ???
    combiner

```

-- Recursive step: join employees to the CTE

SELECT e.id, e.name, e.manager_id,

ot.depth + 1

FROM employees e

org_tree ot **ON** e.manager_id = ot.

join ke...

join col

)

SELECT name, depth

FROM org_tree

ORDER BY depth, name;

Check Answer

Hint

Reveal

Reset

SQL CTEs Tutorial · Name your logic, tame your queries.